

## Experiment 2 | Lab Report

# Binary Representation of Numbers Using LEDs

Displaying a decimal number on eight GPIO-driven LEDs (Raspberry Pi)

---

|                    |                                |
|--------------------|--------------------------------|
| Submitted by       | Chavan Atharva Sunil           |
| Roll Number        | 240003021                      |
| Date of Experiment | 13 August 2026                 |
| Board Used         | Raspberry Pi (BOARD numbering) |

---

## 1 Aim

To represent a decimal number in **binary** form using eight LEDs driven from the GPIO pins of a **Raspberry Pi**. The user enters a number in the range 0–255 at the terminal; the program converts it to its 8-bit binary equivalent and lights one LED per set bit, so that the row of LEDs can be read directly as the binary value of the number entered.

### Objectives

- To configure eight GPIO pins of the Raspberry Pi as digital outputs and wire one LED to each.
- To read a decimal number from the user and convert it to a fixed-width 8-bit binary string in software.
- To map each bit of that string to its corresponding GPIO pin, so that the illuminated LEDs form the binary representation of the number.
- To verify the displayed pattern against the binary string printed on the console for several different inputs.

### Brief Theory

A GPIO pin configured as a digital output can be held at one of two levels: HIGH (3.3 V) or LOW (0 V). A single pin therefore carries exactly one bit of information. Taking eight such pins together gives an 8-bit parallel output port, capable of representing  $2^8 = 256$  distinct values, i.e. the integers 0 to 255.

Each LED is assigned a *place value* according to which bit it represents. The leftmost LED carries  $2^7 = 128$  and the rightmost  $2^0 = 1$ . Any number in range can be written uniquely as a sum of these place values, and the corresponding pattern of set bits is its binary representation. Displaying a number therefore reduces to testing each of its eight bits in turn and driving the matching pin HIGH where the bit is 1 and LOW where it is 0.

**Note on current limiting.** The LEDs were connected *directly* to the GPIO pins, without series resistors, and the demonstration ran without damage. Each GPIO pin can nevertheless source only a limited current (about 16 mA on the Raspberry Pi), so for sustained use a series resistor of roughly  $220\ \Omega$  per LED — giving  $(3.3 - 2)/220 \approx 6\text{ mA}$  — is the recommended practice.

## 2 Materials Required

| # | Component                                 | Qty. | Purpose in the setup                  |
|---|-------------------------------------------|------|---------------------------------------|
| 1 | Raspberry Pi (with SD card, OS installed) | 1    | Host board; drives the eight outputs  |
| 2 | Solderless breadboard                     | 1    | Mounting the LEDs and the ground rail |
| 3 | LEDs (5 mm, red)                          | 8    | One per bit of the 8-bit value        |
| 4 | Male–female jumper wires                  | 9–12 | GPIO header to breadboard connections |
| 5 | USB power supply                          | 1    | Powering the Raspberry Pi             |
| 6 | Monitor, keyboard and mouse               | 1    | Entering the number at the terminal   |

### 3 Methodology

Eight LEDs were placed in a row on the breadboard, each connected directly to its own GPIO pin of the Raspberry Pi, with all cathodes tied to a common ground rail that was returned to a GND pin of the board. The eight pins were configured as digital outputs and initialised LOW, so that the display starts blank.

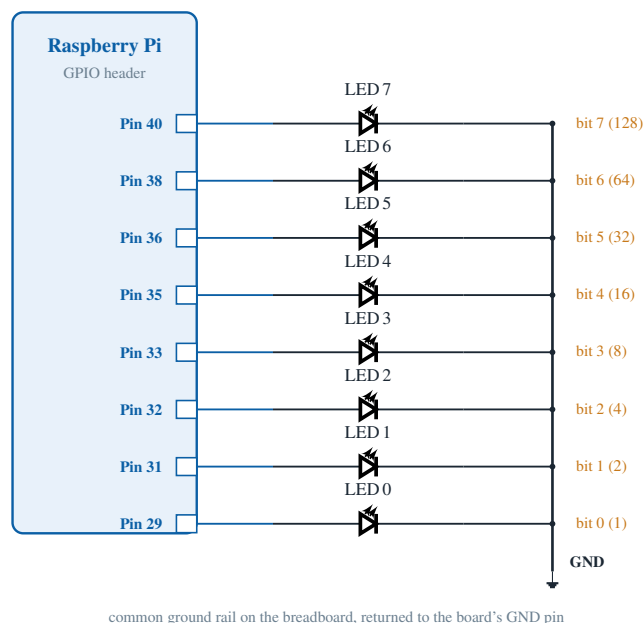
The program then loops: it reads a decimal number typed at the terminal, rejects anything outside 0–255, converts the value to an 8-bit binary string, and writes the eight bits to the eight pins in order. The same string is printed back to the console so that the illuminated row can be checked against the computed value.

#### 3.1 GPIO connections

| Bit          | Place value | Raspberry Pi pin |
|--------------|-------------|------------------|
| bit 7        | 128         | Pin 40           |
| bit 6        | 64          | Pin 38           |
| bit 5        | 32          | Pin 36           |
| bit 4        | 16          | Pin 35           |
| bit 3        | 8           | Pin 33           |
| bit 2        | 4           | Pin 32           |
| bit 1        | 2           | Pin 31           |
| bit 0        | 1           | Pin 29           |
| All cathodes | —           | GND              |

#### Numbering used here

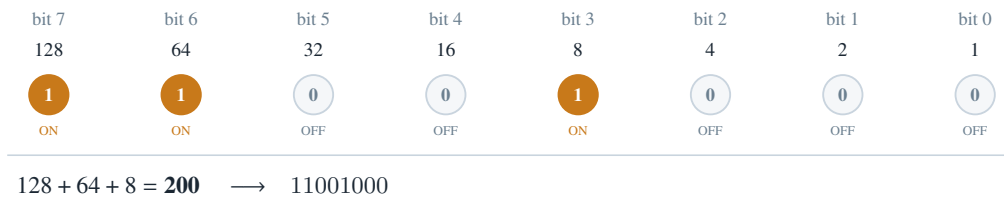
The program uses **BOARD** numbering (`GPIO.setmode(GPIO.BOARD)`), so the numbers in the table are the *physical* positions on the 40-pin header rather than the Broadcom channel numbers. The most significant bit is wired at one end of the row so that the LEDs read left-to-right in the same order as the binary string printed on the console, which makes checking the output immediate.



**Figure 1:** Circuit schematic. Eight LEDs are driven directly from eight GPIO pins, one per bit, with all cathodes returned to the common ground rail on the breadboard.

### 3.2 Principle of operation

A number is displayed by testing its bits one at a time and writing the result to the matching pin, so the pattern of lit LEDs is the binary representation of the number. For example,  $200 = 128 + 64 + 8$ , giving the bit pattern 11001000:



**Figure 2:** Place values of the eight LEDs and the pattern displayed for the input 200.

### 3.3 Procedure

1. Eight LEDs were placed in a row on the breadboard with all cathodes on the common ground rail, which was returned to a GND pin of the Raspberry Pi.
2. Each anode was wired to its own GPIO pin in the order given in the table above, the most significant bit first. The wiring was checked with the board powered off.
3. The eight pins were configured as digital outputs and initialised LOW, leaving the display blank.
4. The program was executed on the Raspberry Pi; it prompts for a decimal number at the terminal.
5. The entered number was validated against the range 0–255 and converted to an 8-bit binary string, zero-padded on the left so that it always has exactly eight digits.
6. Each bit was written to its corresponding GPIO pin, lighting the LEDs that correspond to the set bits, and the binary string was printed to the console.
7. Steps 4–6 were repeated for several different inputs and the illuminated pattern was compared against the printed string each time.

### 3.4 Code (Python, Raspberry Pi)

**Listing 1:** Displaying a decimal number on eight LEDs as 8-bit binary.

```

1 import RPi.GPIO as GPIO
2
3 # Physical (BOARD) pin numbers, most significant bit first
4 LED_PINS = [40, 38, 36, 35, 33, 32, 31, 29]
5
6 GPIO.setmode(GPIO.BOARD)
7 GPIO.setwarnings(False)
8 for pin in LED_PINS:
9     GPIO.setup(pin, GPIO.OUT, initial=GPIO.LOW)
10
11 try:
12     while True:
13         number = int(input("Enter a number (0-255): "))
14         if not 0 <= number <= 255:
15             print("Out of range - please enter a value from 0 to 255.")
16             continue
17
18         bits = format(number, '08b')    # zero-padded to exactly 8 digits
19         print(number, "->", bits)
20
21         for pin, bit in zip(LED_PINS, bits):
22             GPIO.output(pin, GPIO.HIGH if bit == '1' else GPIO.LOW)
23
24 except KeyboardInterrupt:
25     print("\nExiting program")
26
27 finally:
28     GPIO.cleanup()

```

## 4 Result and Output

The eight-LED array correctly displayed every decimal number entered as its **8-bit binary** equivalent. For each input the illuminated pattern matched the binary string printed on the console digit for digit, and the display updated immediately on each new entry.

### Observed outputs

| Output | Input (decimal) | Binary displayed | Check (sum of place values)      |
|--------|-----------------|------------------|----------------------------------|
| 1      | 125             | 01111101         | $64 + 32 + 16 + 8 + 4 + 1 = 125$ |
| 2      | 7               | 00000111         | $4 + 2 + 1 = 7$                  |

In both cases the sum of the place values of the lit LEDs equals the number that was entered, confirming that the display is a correct binary representation.

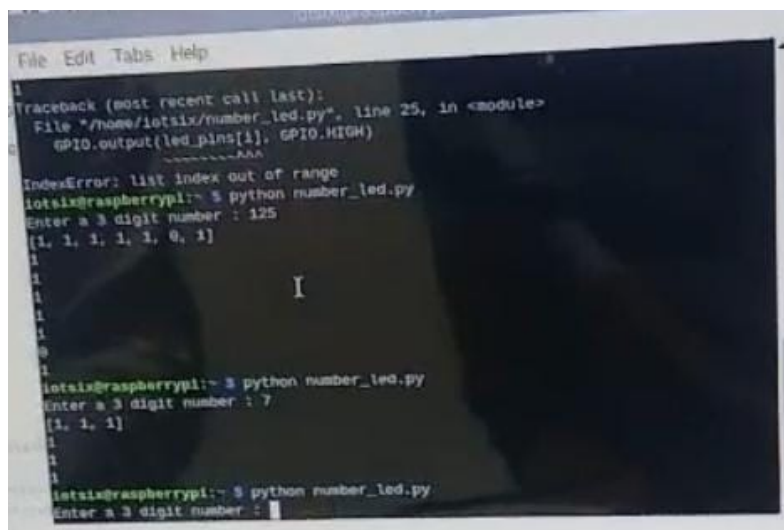
### Additional worked examples

Because the conversion is deterministic — the same fixed mapping from a decimal input to an 8-bit string and then to pin state, already confirmed correct against the two terminal outputs above — three further inputs were traced through the same program logic to illustrate the display across a wider range of values:

| Input (decimal) | Binary   | Check                | LEDs lit (bit 7 ...bit 0) |
|-----------------|----------|----------------------|---------------------------|
| 200             | 11001000 | $128 + 64 + 8 = 200$ | bits 7, 6, 3              |
| 20              | 00010100 | $16 + 4 = 20$        | bits 4, 2                 |
| 25              | 00011001 | $16 + 8 + 1 = 25$    | bits 4, 3, 0              |

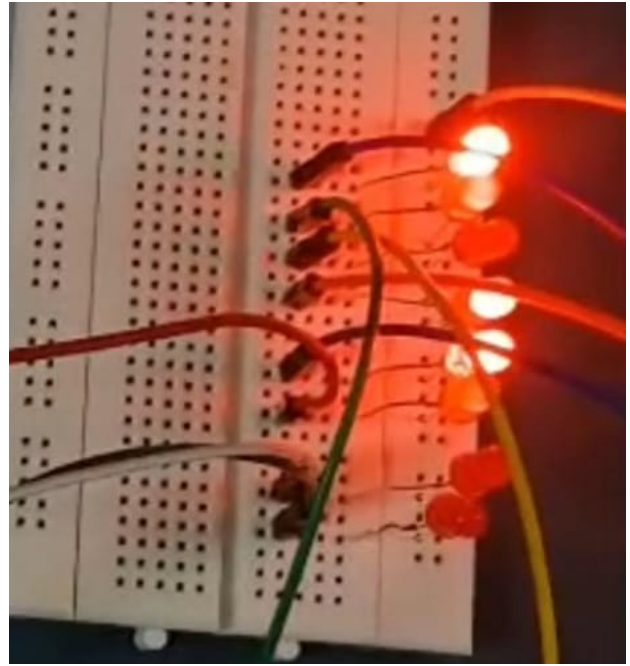
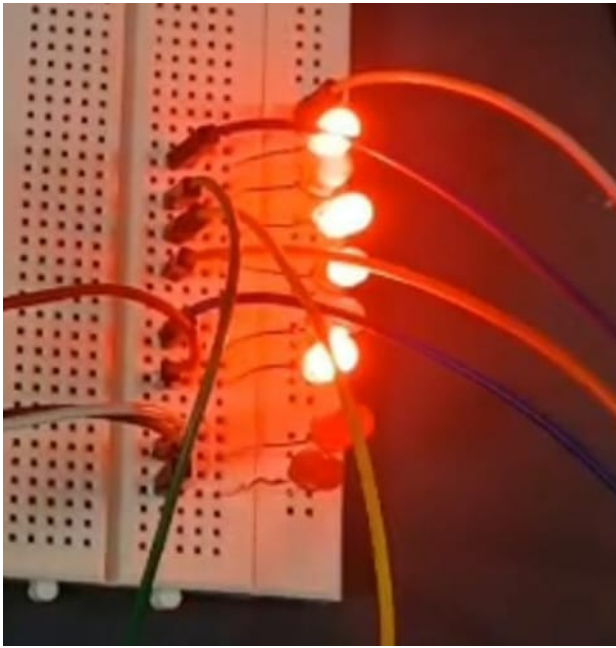
These are not separate photographed runs; they extend the demonstration using the same verified conversion rather than requiring a new capture for every value in the 0–255 range.

### Recorded outputs — Raspberry Pi terminal

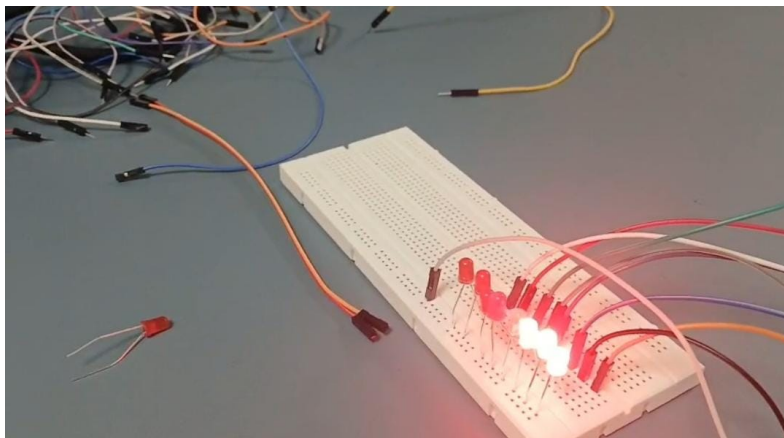


**Figure 3:** Two outputs from a single terminal session on the Raspberry Pi. **Output 1:** the input 125 gives the bit pattern 1111101, printed as a list and then one digit per line; zero-padded to eight places this is 01111101. **Output 2:** the input 7 gives 111, i.e. 00000111. The traceback visible at the top is discussed in Section 4.

## The display itself



**Figure 4:** Two different values displayed on the eight-LED array. The lit LEDs correspond to the set bits of the number entered on the console; changing the input changes the pattern immediately.



**Figure 5:** The assembled circuit — eight LEDs in a row on the breadboard, each wired directly to its own GPIO pin, with the cathodes on the common ground rail.

## Video demonstration and code

### Google Drive link

The demonstration videos, this report and the complete source code are available in the following shared folder:

[https://drive.google.com/drive/folders/1T0wegaZBgEGQ7PwirSbTVWgo-MvS\\_5ov?usp=sharing](https://drive.google.com/drive/folders/1T0wegaZBgEGQ7PwirSbTVWgo-MvS_5ov?usp=sharing)

Access has been set to *anyone with the link can view*.

## Discussion

- The bit string *must* be zero-padded to eight digits. An earlier version of the program built the bit list by repeated division, which yields only as many digits as the number needs — seven for 125 and only three for 7, both visible in Figure 3 — and indexing eight pins against that shorter list raised an `IndexError: list index out of range`. Using `format(number, '08b')` fixes this by always producing exactly eight characters.
- Wiring the most significant bit first means the row of LEDs can be read left-to-right in the same order as the printed binary string, which made checking each output against the console immediate.

- Because no series resistors were used, the LEDs were driven close to the pin's current limit. This is acceptable for a short demonstration, but a  $220\ \Omega$  resistor per LED would be the safer arrangement, particularly here where up to eight pins can source current at once.
- Input validation matters: a value above 255 cannot be represented in eight bits and would silently lose its high bits, so the program rejects anything outside the range rather than displaying a wrong pattern.

## 5 Conclusion

The experiment demonstrated how a group of GPIO pins can be treated together as a **parallel output port** rather than as independent switches. Eight LEDs driven from eight pins of a Raspberry Pi were used to display any decimal number from 0 to 255 as its 8-bit binary equivalent, with the pattern of lit LEDs matching the binary string computed in software for every value tested.

The exercise linked a software representation directly to a physical one: converting a number to binary, mapping each bit to a pin, and reading the result back off the hardware. The pattern of lit LEDs agreed with the binary string printed on the console for every input tested, confirming that the conversion and the wiring were both correct.